KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

WIRELESS ENVIRONMENTAL PARAMETER MONITORING SYSTEM FOR SMART INDUSTRIES

EXERCISE NO – 11

HARRISH T M – 24MER016

KAPIL P – 24MER022

RAMPRASATH D – 24MER047

SARAVANAKARTHIK S R – 24MER055

SETHUPATHI M – 24MER056

11. Wireless Environmental Parameter Monitoring System for Smart Industries

1.Project overview:

The Wireless Environmental Parameter Monitoring System for Smart Industries is an IoT-based system designed to continuously monitor important environmental conditions in industrial environments. The system uses an ESP8266 NodeMCU as the main controller, a DHT11/DHT22 sensor to measure temperature and humidity, and an MQ135 sensor to monitor air quality.

The collected sensor data is processed by the ESP8266 and transmitted wirelessly through Wi-Fi to Adafruit IO, where it can be viewed remotely using a dashboard. The system also uses a buzzer to provide a local warning when the detected air-quality level exceeds the defined threshold. This enables early detection of unhealthy environmental conditions and supports safer and more efficient industrial operation.

2.Problem Statement:

Industrial environments can experience changes in temperature, humidity, and air quality due to machinery, chemical processes, dust, and gas emissions. Conventional monitoring methods may require manual inspection and may not provide continuous or remote monitoring. This can delay the detection of unsafe environmental conditions and affect worker safety, equipment performance, and industrial productivity. Therefore, there is a need for a wireless and real-time environmental monitoring system that can continuously measure temperature, humidity, and air quality, provide immediate alerts during abnormal conditions, and allow the parameters to be monitored remotely through an IoT platform.

3.Proposed Solution:

The proposed system uses an ESP8266 NodeMCU to develop a wireless environmental monitoring system for smart industries. A DHT11/DHT22 sensor continuously measures temperature and humidity, while an MQ135 sensor monitors air-quality levels. The ESP8266 processes the sensor data and sends it wirelessly through Wi-Fi to Adafruit IO for real-time remote monitoring. A buzzer is activated when the air-quality value exceeds the predefined threshold, providing an immediate local warning. This system enables continuous monitoring, early detection of unsafe conditions, and improved environmental safety in industrial environments.

4.System Architecture:

1. **Environmental Sensing Layer**
    - **DHT11/DHT22** measures temperature and humidity.
    - **MQ135** detects changes in air quality and gas concentration.
2. **Processing Layer**
    - **ESP8266 NodeMCU** receives the sensor readings.
    - It processes the data and compares the MQ135 value with predefined air-quality thresholds.
3. **Alert Layer**
    - A **buzzer** provides an immediate warning when poor air quality is detected.
4. **Communication Layer**
    - ESP8266 uses **Wi-Fi** to transmit the collected environmental data.
5. **Cloud Monitoring Layer**
    - **Adafruit IO** receives and stores the sensor readings.
    - Temperature, humidity, air-quality value, and air-quality status can be monitored remotely through a dashboard.
6. **User Monitoring Layer**
    - The user views the real-time environmental parameters through the Adafruit IO dashboard.
    - Abnormal air-quality conditions can be identified quickly for appropriate action.

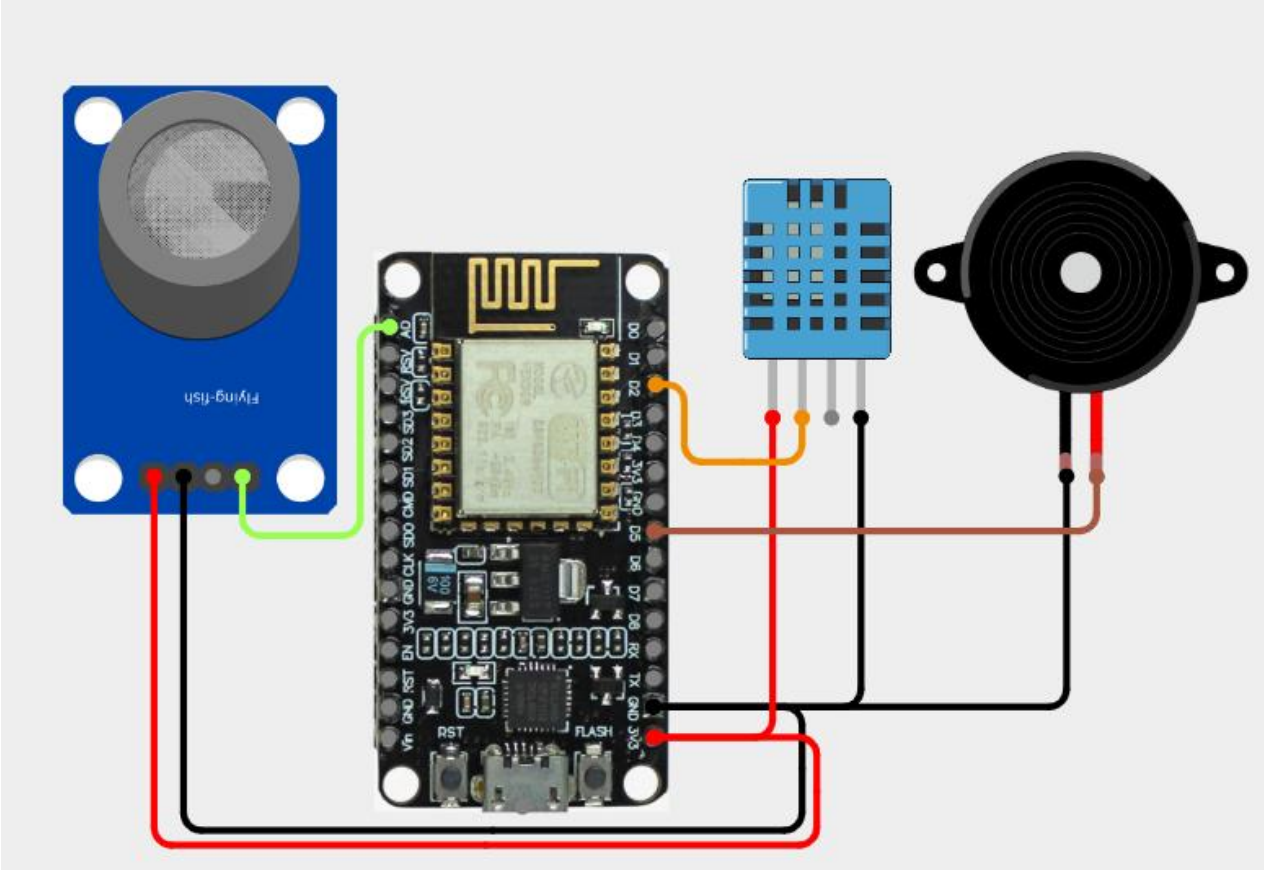
5.Circuit Table

COMPONENT	PIN	CONNECTION
DHT11 Sensor	VCC	3.3 V
DHT11 Sensor	DATA	D2
DHT11 Sensor	GND	GND
MQ2	VCC	VIN (5V)
MQ2	GND	GND
MQ2	AO	A0
Buzzer	+	D5
Buzzer	GND	GND

6. Circuit Design:

The circuit is designed using an ESP8266 NodeMCU, DHT11/DHT22, MQ135 air-quality sensor, and buzzer. The ESP8266 acts as the central controller and processes the sensor data.

- The DHT11/DHT22 is connected to D2 (GPIO4) to measure temperature and humidity.
- The MQ135 analog output (AO) is connected to A0 to obtain the air-quality value.
- The MQ135 digital output (DO) is not used because the system requires the actual variable air-quality reading.
- The buzzer is connected to D5 (GPIO14) and provides an alert when the air-quality value exceeds the predefined threshold.
- All sensor grounds are connected to the ESP8266 GND.
- The DHT sensor is powered from 3.3 V, while the MQ135 module can be powered from VIN/5 V according to the module requirements.
- The ESP8266 uses its built-in Wi-Fi to transmit the measured parameters to Adafruit IO for remote monitoring.



7. Algorithm:

1. Start the system.
2. Initialize the ESP8266, DHT11/DHT22, MQ135, and buzzer.
3. Connect the ESP8266 to the configured Wi-Fi network.
4. Establish communication with Adafruit IO.
5. Read temperature and humidity from the DHT11/DHT22 sensor.
6. Read the analog air-quality value from the MQ135 sensor.
7. Compare the MQ135 value with the predefined thresholds:
    - 0–399 → GOOD
    - 400–599 → MODERATE
    - 600–1023 → POOR
8. If the air quality is POOR, turn ON the buzzer; otherwise, keep it OFF.
9. Send temperature, humidity, air-quality value, and air-quality status to Adafruit IO.
10. Display the parameters on the Adafruit IO dashboard.
11. Repeat the process continuously for real-time monitoring.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>

#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"

#define IO_USERNAME "saravanakarthiksr"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

#define DHT_PIN     4
#define MQ135_PIN   A0
#define BUZZER_PIN  14
```

```
#define DHTTYPE DHT11

DHT dht(DHT_PIN, DHTTYPE);

AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
AdafruitIO_Feed *humidityFeed    = io.feed("humidity");
AdafruitIO_Feed *airQualityFeed  = io.feed("air-quality");
AdafruitIO_Feed *airStatusFeed   = io.feed("air-status");


#define GOOD_LIMIT       399
#define MODERATE_LIMIT   599

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("================================================");
  Serial.println(" WIRELESS ENVIRONMENT MONITORING SYSTEM");
  Serial.println(" ESP8266 + DHT + MQ135 + BUZZER");
  Serial.println(" ADafruit IO");
  Serial.println("================================================");
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Adafruit IO Connected!");
  Serial.println("------------------------------------------");
}

void loop()
```

```
{
 io.run();

 float temperature = dht.readTemperature();
 float humidity = dht.readHumidity();

 if (isnan(temperature) || isnan(humidity))
 {
  Serial.println("ERROR: DHT sensor reading failed!");
  delay(2000);
  return;
 }

 int airQualityValue = analogRead(MQ135_PIN);

 String airStatus;

 if (airQualityValue <= GOOD_LIMIT)
 {
  airStatus = "GOOD";

  digitalWrite(BUZZER_PIN, LOW);
 }
 else if (airQualityValue <= MODERATE_LIMIT)
 {
  airStatus = "MODERATE";

  digitalWrite(BUZZER_PIN, LOW);
 }
 else
 {
  airStatus = "POOR";

  digitalWrite(BUZZER_PIN, HIGH);
 }

 Serial.println();
 Serial.println("---------- SENSOR DATA ----------");

 Serial.print("Temperature       : ");
 Serial.print(temperature);
 Serial.println(" °C");

 Serial.print("Humidity          : ");
```

```
  Serial.print(humidity);
  Serial.println(" %");

  Serial.print("Air Quality Value : ");
  Serial.println(airQualityValue);

  Serial.print("Air Quality       : ");
  Serial.println(airStatus);

  Serial.print("Buzzer            : ");

  if (airQualityValue > MODERATE_LIMIT)
  {
    Serial.println("ON");
  }
  else
  {
    Serial.println("OFF");
  }

  Serial.println("----------------------------------------");

  Serial.println("Sending data to Adafruit IO...");

  temperatureFeed->save(temperature);
  humidityFeed->save(humidity);
  airQualityFeed->save(airQualityValue);
  airStatusFeed->save(airStatus);

  Serial.println("Data sent successfully!");

  delay(5000);
}
```

9. System Working:

1. The ESP8266 NodeMCU initializes the DHT11/DHT22, MQ135 sensor, and buzzer.
2. The DHT11/DHT22 continuously measures the surrounding temperature and humidity.
3. The MQ135 detects changes in the surrounding air and provides an analog air-quality value through its AO pin.
4. The ESP8266 reads and processes the sensor values.
5. The MQ135 value is compared with predefined thresholds to classify the air quality as GOOD, MODERATE, or POOR.
6. When the air quality reaches the POOR level, the buzzer is activated as an alert.
7. The ESP8266 connects to the Internet through Wi-Fi and sends the measured values to Adafruit IO.
8. The Adafruit IO dashboard displays temperature, humidity, air-quality value, and air-quality status for remote monitoring.
9. The system continuously repeats these operations, enabling real-time environmental monitoring for smart industries.

```
------------- SENSOR DATA -------
Temperature       : 28.80 °C
Humidity          : 77.00 %
Air Quality Value : 4
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!

------------- SENSOR DATA -------
Temperature       : 28.90 °C
Humidity          : 84.70 %
Air Quality Value : 0
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!

------------- SENSOR DATA -------
Temperature       : 29.30 °C
Humidity          : 88.60 %
Air Quality Value : 4
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!
```

```
------------- SENSOR DATA -------
Temperature       : 28.10 °C
Humidity          : 73.00 %
Air Quality Value : 0
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!

------------- SENSOR DATA -------
Temperature       : 28.10 °C
Humidity          : 72.90 %
Air Quality Value : 4
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!

------------- SENSOR DATA -------
Temperature       : 28.10 °C
Humidity          : 72.90 %
Air Quality Value : 4
Air Quality       : GOOD
Buzzer            : OFF
---------------------------------
Sending data to Adafruit IO...
Data sent successfully!
```

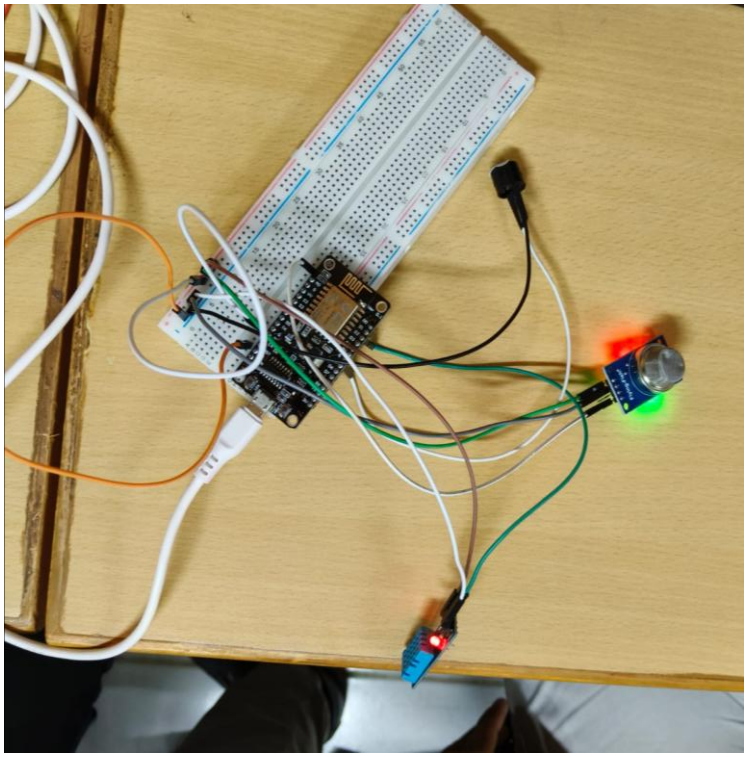
10. Bill of Materials:

COMPONENT	QUANTITY
ESP8266 NodeMCU	1
DHT11 Sensor	1
MQ2 Sensor	1
Buzzer	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:

The prototype was implemented using an ESP8266 NodeMCU as the main controller, with a DHT11/DHT22 sensor for temperature and humidity measurement and an MQ135 sensor for air-quality monitoring. The sensors were connected to the ESP8266 through a breadboard, and a buzzer was added to provide an alert when the air-quality value exceeded the predefined threshold.

The ESP8266 was programmed using the Arduino IDE to read, process, and classify the sensor data into GOOD, MODERATE, and POOR air-quality levels. The controller was connected to a Wi-Fi network, and the measured temperature, humidity, air-quality value, and status were transmitted to Adafruit IO. A dashboard was created to display the parameters remotely in real time. The prototype successfully demonstrates continuous environmental monitoring and immediate warning during poor air-quality conditions.



12.Results & Performance Analysis:

The developed system successfully monitored the environmental conditions using the ESP8266 NodeMCU. The DHT11/DHT22 sensor measured temperature and humidity continuously. The MQ135 sensor monitored the air-quality level and provided an analog value. The ESP8266 processed all sensor readings in real time.

The air-quality value was classified into GOOD, MODERATE, and POOR levels based on predefined thresholds. When poor air quality was detected, the buzzer was automatically activated. The system provided an immediate local alert to indicate unsafe air conditions. The ESP8266 connected successfully to the Wi-Fi network. Sensor readings were transmitted to Adafruit IO for remote monitoring.

Temperature, humidity, air-quality value, and air-quality status were displayed on the dashboard. The system provided continuous monitoring without requiring manual measurement. The wireless communication enabled the user to monitor industrial conditions remotely. The prototype showed quick response to changes in air quality.

The system operated with simple and low-cost components. It can help in identifying abnormal environmental conditions at an early stage. The collected data can also be used for further analysis and improvement. Overall, the system achieved its objective of wireless environmental monitoring.

The prototype demonstrates the usefulness of IoT technology for smart industrial environments.